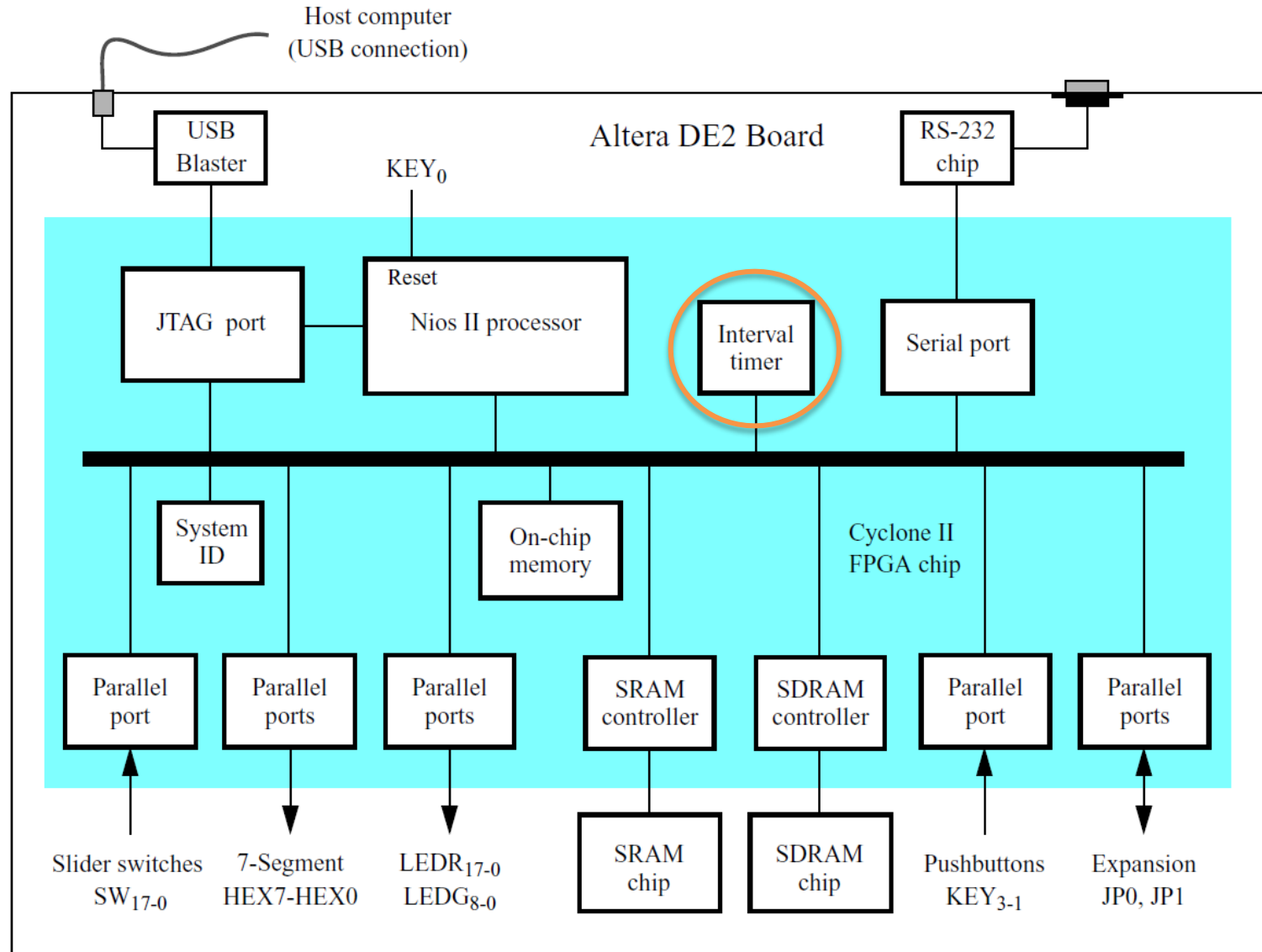


Interval Timer Core sous HAL

Avalon IP Core Slave
Interval timer core

Exemple de SoC : DE0-nano Basic Computer



Avalon IP Core Slave

Interval timer core

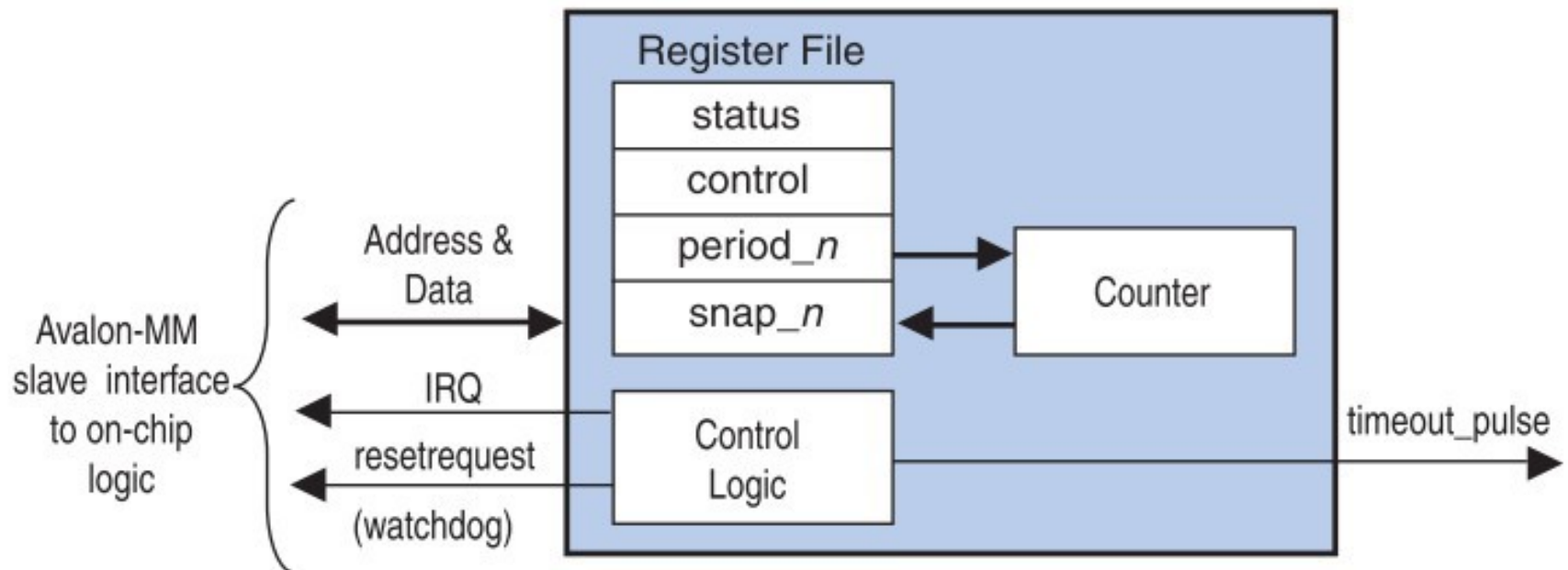
- Configuration Hardware :
 - **32-bit or 64-bit** **compteur interne**
 - Deux modes de comptage: count down once et continuous count- down; valeur de départ chargée à partir de period register
 - Option permettant d'activer ou de désactiver la demande d'interruption (IRQ) quand le comptage atteint zéro
 - **Option** Watchdog, fonctionnalité qui réinitialise le système si le compteur atteint zéro
 - **Option** Générateur d'impulsions périodiques, fonctionnalité générant une Impulsion lorsque le compteur atteint zéro
- Device driver disponible dans la **librairie HAL**

Avalon IP Core Slave

Interval timer core

- **Bloc diagramme**

- 6x (compteur 32-bits) ou 10x (compteur 64-bits)
- **Registres 16-bits** (Certains registres peuvent ne pas être présents en fonction de la configuration principale)



Avalon IP Core Slave

Interval timer core

- **Registre de contrôle** modifié par le Nios II pour :
 - Démarrer et arrêter le compteur
 - Enable/disable l'IRQ
 - Spécifier le mode count-down once ou continuous count-down
- La **lecture du registre d'état** déduit le temps d'activité
- Un processeur peut spécifier la période du timer par **l'écriture de la valeur dans period register**
 - Un compteur interne compte à rebours jusqu'à zéro, et chaque fois qu'il atteint zéro, il est immédiatement rechargé à partir des registres **period register**.
Un processeur peut lire la valeur du compteur actuel via **snap registers**
- Lorsque le décomptage atteint zéro, un ou plusieurs des événements suivants sont déclenchés :
 - Si les IRQs sont activés, une IRQ est générée
 - L'option pulse-generator génère une impulsion
 - L'option watchdog génère un reset pour réinitialiser le système

Avalon IP Core Slave

Interval timer core

- Configuration d'une l'instance via QSys

The image displays the QSys configuration interface for the Altera Avalon Timer IP core. The interface is divided into two main sections: the Block Diagram on the left and the configuration parameters on the right.

Block Diagram: The diagram shows the 'Interval_timer' block (labeled 'altera_avalon_timer') with the following connections:

- clk:** Connected to the 'clk' input of the timer block.
- reset_n:** Connected to the 'reset_n' input of the timer block.
- address[2..0]:** Connected to the 'address' input of the timer block.
- writedata[15..0]:** Connected to the 'writedata' input of the timer block.
- readdata[15..0]:** Connected to the 'readdata' output of the timer block.
- chipselect:** Connected to the 'chipselect' input of the timer block.
- write_n:** Connected to the 'write_n' input of the timer block.
- irq:** The timer block has an 'irq' output, which is connected to an external 'irq' signal.

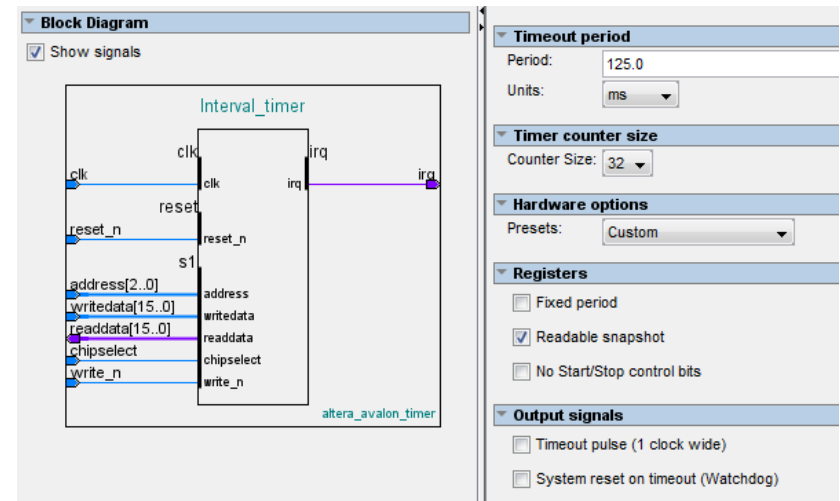
Configuration Parameters:

- Timeout period:**
 - Period: 125.0
 - Units: ms
- Timer counter size:**
 - Counter Size: 32
- Hardware options:**
 - Presets: Custom
- Registers:**
 - ☐ Fixed period
 - ☒ Readable snapshot
 - ☐ No Start/Stop control bits
- Output signals:**
 - ☐ Timeout pulse (1 clock wide)
 - ☐ System reset on timeout (Watchdog)

Avalon IP Core Slave

Interval timer core

- Timeout period: détermine la valeur initiale de **period registers**; peut être modifié en fonction de fixed period option
- Taille du compteur: 32- or 64-bits
- Options Hardware : 3 Configurations prédéfinies
 - Simple periodic interrupt
 - Watchdog
 - Ou custom



Avalon IP Core Slave

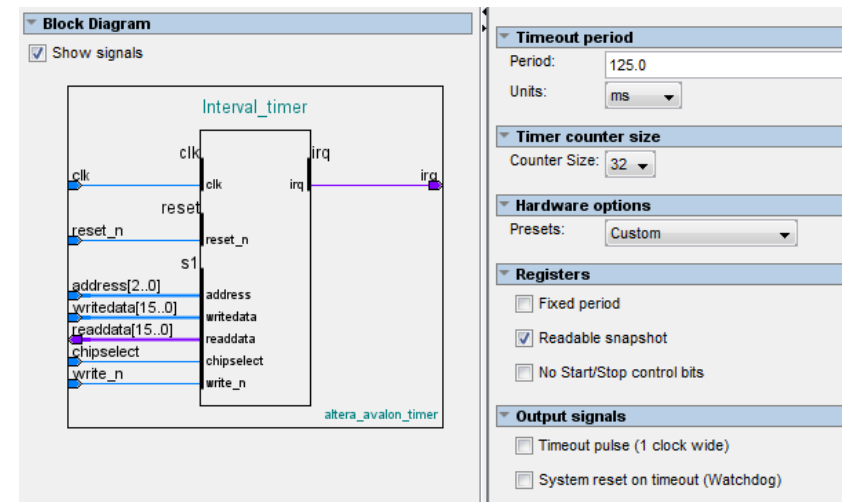
Interval timer core

- **Options Registre**

- Writeable period
- Readable snapshot
- Start/Stop control bits

- **Signaux de sortie**

- Timeout pulse (1 clock wide)
- Réinitialisation du système lors du TimeOut (**watchdog**)



Avalon IP Core Slave

Interval timer core

- **Comportement du Watchdog :**
 - Après le reset, le compteur est arrêté
 - Il doit être redémarré en écrivant un 1 dans le bit START du registre de contrôle. **Une fois démarrée, le timer ne peut jamais être arrêté**
 - Si le compteur interne atteint zéro, le watchdog reset le système en générant une impulsion sur sa sortie de demande de réinitialisation
 - Pour empêcher la réinitialisation du système, le processeur/programme doit réinitialiser périodiquement la valeur du compteur en écrivant dans l'un des registres de période (la valeur écrite est ignorée)
 - Si le processeur ne parvient pas à accéder au Timer parce que, par exemple, l'application a cessé de fonctionner normalement, le Timer de surveillance réinitialise le système et le ramène à un état défini

Avalon IP Core Slave

Interval timer core

- Status register

Bit	Name	R/W/C	Description
0	TO	RC	The TO (timeout) bit is set to 1 when the internal counter reaches zero. Once set by a timeout event, the TO bit stays set until explicitly cleared by a master peripheral. Write zero to the <code>status</code> register to clear the TO bit.
1	RUN	R	The RUN bit reads as 1 when the internal counter is running; otherwise this bit reads as 0. The RUN bit is not changed by a write operation to the <code>status</code> register.

Avalon IP Core Slave

Interval timer core

- Control register

Bit	Name	R/W/C	Description
0	ITO	RW	If the ITO bit is 1, the interval timer core generates an IRQ when the status register's TO bit is 1. When the ITO bit is 0, the timer does not generate IRQs.
1	CONT	RW	The CONT (continuous) bit determines how the internal counter behaves when it reaches zero. If the CONT bit is 1, the counter runs continuously until it is stopped by the STOP bit. If CONT is 0, the counter stops after it reaches zero. When the counter reaches zero, it reloads with the value stored in the period registers, regardless of the CONT bit.
2	START (1)	W	Writing a 1 to the START bit starts the internal counter running (counting down). The START bit is an event bit that enables the counter when a write operation is performed. If the timer is stopped, writing a 1 to the START bit causes the timer to restart counting from the number currently stored in its counter. If the timer is already running, writing a 1 to START has no effect. Writing 0 to the START bit has no effect.
3	STOP (1)	W	Writing a 1 to the STOP bit stops the internal counter. The STOP bit is an event bit that causes the counter to stop when a write operation is performed. If the timer is already stopped, writing a 1 to STOP has no effect. Writing a 0 to the stop bit has no effect. If the timer hardware is configured with Start/Stop control bits off, writing the STOP bit has no effect.

Avalon IP Core Slave

Interval timer core

- **period_n Registers**

- The **period_n** registers stocke la valeur du timeout
- Internal counter est chargé avec la valeur stockée dans ces registres chaque fois que l'une des situations suivantes se produit :
 - Une opération d'écriture sur l'un des **period_n** register
 - **Internal counter** atteint la valeur 0
- En écrivant sur l'un des registres **period_n** arrête **internal counter**, sauf lorsque le timer est configuré avec un Start/Stop control bits à off
- Lorsque le matériel est configuré avec la période accessible en écriture désactivée, l'écriture dans **period_n initialise** the compteur à la valeur fixée par la configuration initiale
- La période réelle est supérieure d'un cycle à la valeur stockée dans le **period_n** registers

Avalon IP Core Slave

Interval timer core

- **snap_n** registers
 - Un périphérique maître peut demander un instantané cohérent du compteur interne actuel en effectuant une opération d'écriture (données d'écriture ignorées) sur l'un des **snap_n** registers
 - Lorsqu'une écriture se produit, la valeur du compteur est copiée dans **snap_n** registers

Avalon IP Core Slave

Interval timer core

- Interrupt Behaviour

- Interval timer core génère une IRQ chaque fois que le compteur interne atteint zéro et que le bit ITO du registre de contrôle est positionné à 1
- L'acquittement de l'IRQ se fait de deux manières :
 - Mise à zéro du bit TO de status register
 - Désactivez les interruptions en effaçant le bit ITO bit du registre de contrôle
- Le fait de ne pas accuser réception à l'IRQ produit un résultat non défini

Avalon IP Core Slave

Interval timer core

Modèle de programmation

HAL
(Custom Device)



sys/alt_alarm.h



altera_avalon_timer_sc.c



sys/alt_timestamp.h



altera_avalon_timer_tc.c

**Device
Driver**



altera_up_avalon_timer.h



altera_up_avalon_timer_regs.h

-----  system.h

**Interval timer
core**

Address - base		31	15	0
0		STATUS		
4		CONTROL		
8		PERIOD_L		
12		PERIOD_H		
16		SNAP_L		
20		SNAP_H		

Avalon IP Core Slave

Interval timer core

Modèle de programmation

- Le choix du modèle de fonctionnement du timer peut être choisi par l'intermédiaire du **BSP**
- Cette propriété est enregistrée dans **system.h**

```
/*  
 * hal configuration  
 *  
 */
```

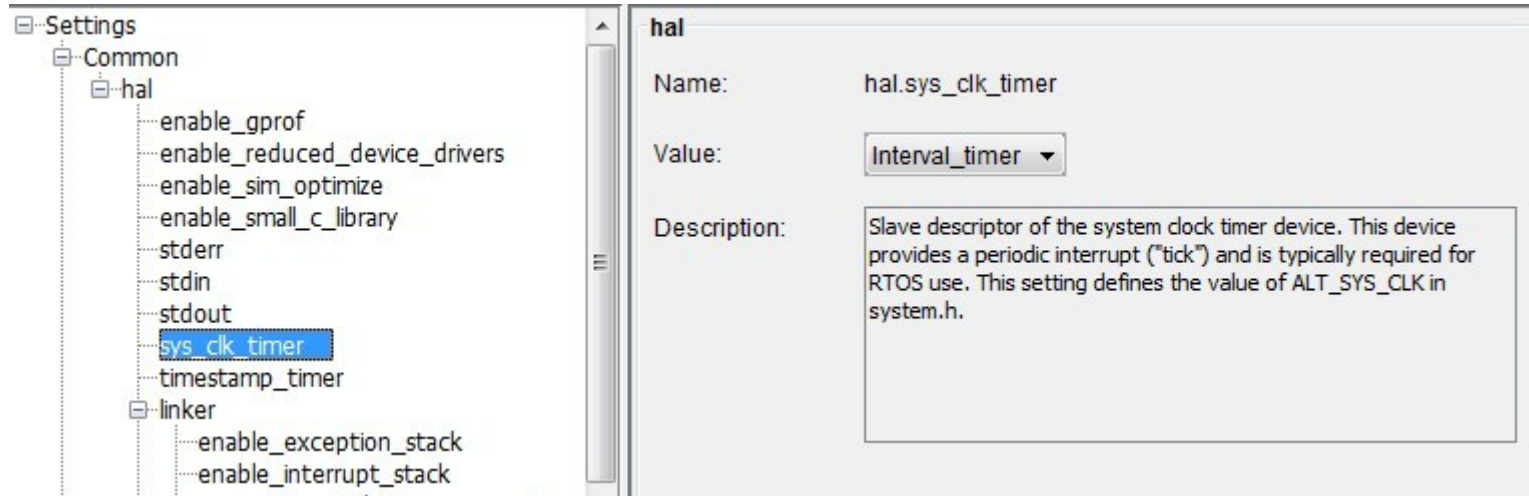
```
#define ALT_MAX_FD 32  
#define ALT_SYS_CLK INTERVAL_TIMER  
#define ALT_TIMESTAMP_CLK none
```

Avalon IP Core Slave

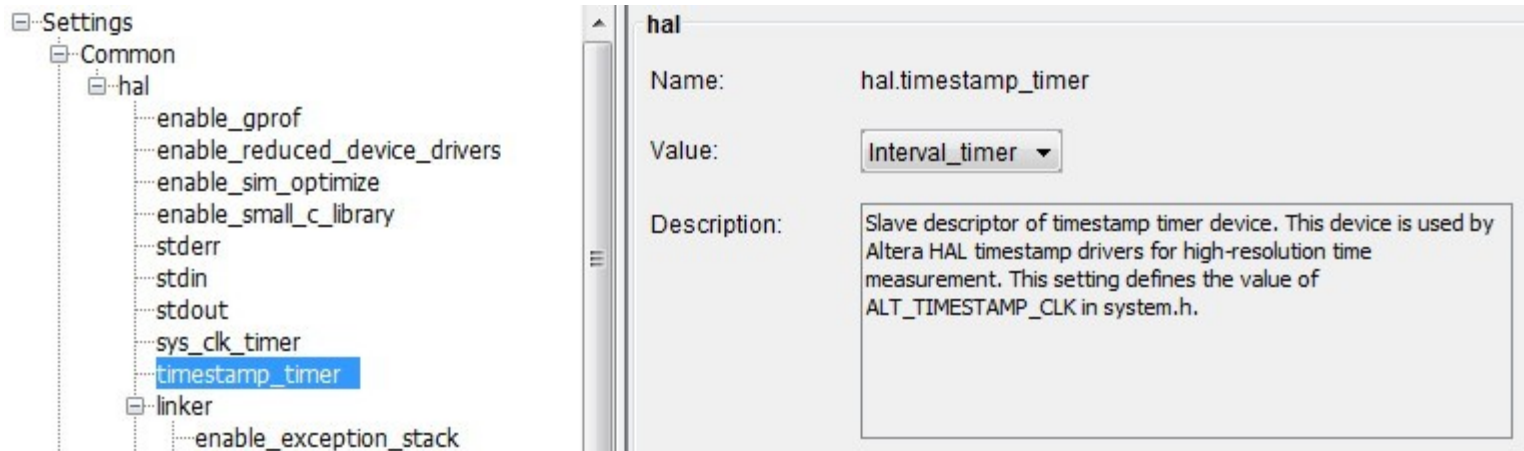
Interval timer core

Modèle de programmation

- HAL/sys_clk_timer lié à Interval_timer



- HAL/timestamp_timer lié à Interval_timer



Avalon IP Core Slave

Interval timer core

L'api HAL implémente les fonctions standard UNIX associées. On cite :

- gettimeofday()
- settimeofday()
- times()

Les temps renvoyés par ces fonctions sont basés sur l'horloge du système.

Avalon IP Core Slave

Interval timer core

- Propriétés enregistrées dans **system.h**

```
/*
```

```
 * hal configuration
```

```
 *
```

```
*/
```

```
#define
```

```
ALT_MAX_FD 32
```

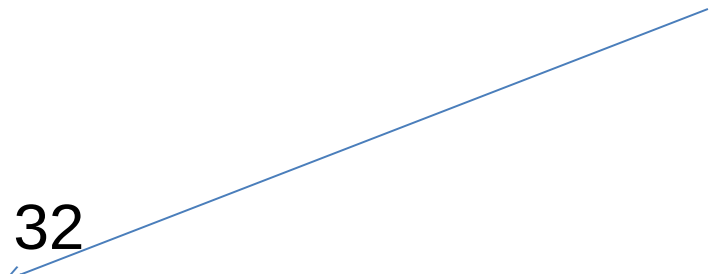
```
#define
```

```
ALT_SYS_CLK INTERVAL_TIMER
```

```
#define
```

```
ALT_TIMESTAMP_CLK none
```

System clock driver



Timestamp driver

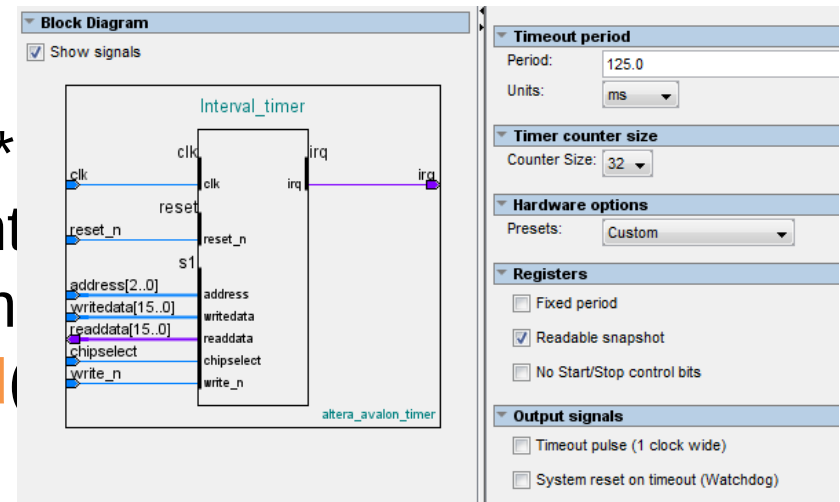


Avalon IP Core Slave

Interval timer core

Fonctions Alarme (HAL) (CONT=0)

- Utiles pour la programmation de tâches périodiques
 - Peut générer des **ticker** (interruptions périodiques)
 - La **periode du ticker** est un multiple de la période choisie pour le timer
 - Pris en charge par « system clock » driver
- Fonctions de base:
 - int **alt_alarm_start**(alt_alarm*
alt_u32 (*callback) (void* cont
 - void **alt_alarm_stop**(alt_alarm
 - alt_u32 **alt_ticks_per_second**(
 - alt_u32 **alt_nticks**(void);



Avalon IP Core Slave

Interval timer core

alt_alarm_start

- int alt_alarm_start(alt_alarm* alarm, alt_u32 nticks, alt_u32 (*callback) (void* context), void* context);

Programme l'appel de la fonction callback tous les nticks

L'argument alarm pointe vers structure en relation avec le pilote (timer) utilisé

La valeur 0 indique le succès de la mise en place de l'alarme

Avalon IP Core Slave

Interval timer core

alt_alarm_stop

```
void alt_alarm_stop(alt_alarm* alarm);
```

Arrête l'appel à la fonction callback initié par alt_alarm_start

Avalon IP Core Slave

Interval timer core

alt_ticks_per_second

```
alt_u32 alt_ticks_per_second(void);
```

Réenvoie le nombre de ticks par seconde associé au Timer

Avalon IP Core Slave

Interval timer core

alt_u32 alt_nticks

alt_u32 alt_nticks(void);

Retourne le nombre de tick écoulée depuis « reset »

Si zéro retourné, aucun système d'horloge n'est disponible

Avalon IP Core Slave

Interval timer core

Exemple – 1 (Système Alarme)

```
#include <stdio.h>
#include <sys/alt_alarm.h>
#include <sys/time.h>
unsigned int tps;

alt_u32 alarm_callback (void* context)
{
    struct timeval now;
    int rc;
    rc = gettimeofday (&now, NULL);
    printf("Seconds: %u\n", (unsigned int)now.tv_sec);
    return tps;
}
```

Avalon IP Core Slave

Interval timer core

```
int main(void)
{
    static alt_alarm alarm;
    struct timeval now;
    int rc;

    tps = alt_ticks_per_second ();
    now.tv_sec = (4*86400)+(23*3600)+(58*60)+50; /*(J,H,M,S)*/
    now.tv_usec = 0;
    rc = settimeofday (&now, NULL);
    alt_alarm_start( &alarm, tps, alarm_callback, NULL
    while(1);

}
```

Avalon IP Core Slave

Interval timer core

```
struct timeval {  
    time_t tv_sec;           /* seconds */  
    suseconds_t tv_usec; /* microseconds */  
};
```

Spécifié dans `<sys/time.h>`

Donne le nombre de secondes et de microsecondes depuis Epoch (1970-01-01 00:00:00 +0000 (UTC))

Epoch (de l'anglais époque ou ère) représente la date initiale

à partir de laquelle est mesuré le temps sur des machines informatiques

Avalon IP Core Slave

Interval timer core

Exemple 2 – Programmation d'alarmes multiples

```
#include <stdio.h>
```

```
#include <sys/alt_alarm.h>
```

```
unsigned int tps;
```

```
alt_u32 alarm_1_callback (void* context)
```

```
{
```

```
    static unsigned int count = 0;
```

```
    printf("alarm 1: %d\n",++count);
```

```
    return tps*2;
```

```
}
```

```
alt_u32 alarm_2_callback (void* context)
```

```
{
```

```
    static unsigned int count = 0;
```

```
    printf("alarm 2: %d\n",++count);
```

```
    return tps*3;
```

```
}
```

Avalon IP Core Slave

Interval timer core

```
int main(void)
{
    alt_alarm alarm_1;
    alt_alarm alarm_2;

    printf("starting alarms\n");
    tps = alt_ticks_per_second();

    alt_alarm_start ( &alarm_1, tps, alarm_1_callback, NULL );
    alt_alarm_start ( &alarm_2, tps, alarm_2_callback, NULL );

    while(1);
}
```

Avalon IP Core Slave

Interval timer core

Exercice « system clock »

Tenant compte de la couche HAL et de « system clock », réaliser une horloge numérique dont l'affichage se fait sur les 7 segments de la carte DE1

Avalon IP Core Slave

Interval timer core

Timestamp HAL (CONT = 1)

- Mesure des intervalles de temps tenant compte de la période d'Horloge du système
- Pris en charge par « timestamp » driver
- **Fonctions de base:**
 - int alt_timestamp_start(void);
 - alt_u32 alt_timestamp(void);
 - alt_u32 alt_timestamp_freq(void);

Avalon IP Core Slave

Interval timer core

`alt_u32 alt_timestamp`

`alt_u32 alt_timestamp(void);`

Retourne la valeur courante de “timestamp counter”

Avalon IP Core Slave

Interval timer core

alt_timestamp_start

```
int alt_timestamp_start(void);
```

Cette fonction démarre le “system timestamp”.
Retourne zéro si succès.

Elle est associée à “timestamp driver”

Avalon IP Core Slave

Interval timer core

```
alt_u32 alt_timestamp_freq(void);
```

```
alt_u32 alt_timestamp_freq(void);
```

Retourne la fréquence de l'incrément du compteur.

Avalon IP Core Slave

Interval timer core

Exemple 3 – Mesure de temps

```
#include <stdio.h>
#include "sys/alt_timestamp.h"
#include "alt_types.h"

int main (void)
{
    alt_u32 time1; alt_u32 time2; alt_u32 time3;
    if (alt_timestamp_start() < 0)
    {
        printf ("No timestamp device available\n");
    }
    else
    {
        time1 = alt_timestamp();
        wait_ms() ; /* first function to monitor */
        time2 = alt_timestamp();
        wait_ms() ; /* second function to monitor */
        time3 = alt_timestamp();
        printf ("Temps 1er Wait = %u ticks\n",(unsigned int)(time2-time1));
        printf ("Temps 2ème Wait= %u ticks\n",(unsigned int)(time3-time2));
        printf ("Ticks per second = %u\n",(unsigned int)
            alt_timestamp_freq());
    }
    return 0;}

```

Avalon IP Core Slave

Interval timer core

Mise en pratique

- Utilisation de **Timestamp HAL** pour :
 - Vérifiez le délai généré par la fonction `usleep()` et affichage du résultat sur JTAG_UART

Avalon IP Core Slave

Interval timer core

Exercice - 1

- Choisir une fonction de votre choix
- L'implémenter en langage c
- Mesurer le temps consommé par cette fonction
- Réaliser la même fonction en Hardware et sous forme d'instruction personnalisée.
- Mesurer son nouveau temps d'exécution.
- Conclure

Avalon IP Core Slave

Interval timer core

Exercice - 2

- A l'aide du Timer étudié ci-avant, développer une fonction réalisant la génération d'un signal PWM
- Fixer le rapport cyclique tenant compte des paramètres utiles du Timer.

Références

- Altera, “Embedded Peripherals IP User Guide,” ***ug_embedded_ip.pdf***
- Altera “Nios II Software Developer’s Handbook,” [n2sw_nii5v2.pdf](#)
 - Chapters 6. Developing Programs Using Hardware Abstraction Layer